SIR DR. M. S. GOSAVI POLYTECHNIC INSTITUTE

Nashik Road, Nashik - 422101

Project name : Blood Bank Management System

Programme Name: Computer Engineering

Academic year: 2026-27

Semester : 3rd

Course Name and Code: OBJECT ORIENTED PROGRAMMING (313304)

MICRO-PROJECT REPORT

Submitted by the group of SYCO-A students

SR. No	Roll no.	Name of Student	Enrollment No.	Seat No.
01	29	ALAI YASH AMEY	25612590030	531279
02	17	PATIL PRATHMESH JITENDRA	25612590018	531267
03	12	PRAVIN CHETAN PATIL	25612590013	531262
04	20	NIKAM ROHIT ANIL	25612590021	531270

Under the Guidance of

Prof. N.B Ghati

Maharashtra State Board of Technical Education,Mumbai

Sir. Dr.M.S.Gosavi Polytechnic Institute Nashik Road

Certificate

This is to certify that following students of **THIRD SEMISTER** Diploma Engineering Program in Computer Engineering have successfully completed the Micro-Project entitled "**Blood Bank Management System**" under my supervision, in the partial fulfillment of Course **OOP (313304)** for Academic Year **2026-27** as per prescribed in the **MSBTE "K Scheme" curriculum.**

SR. No	Roll no.	Name of Student	Enrollment No.	Seat No.
01	29	ALAI YASH AMEY	25612590030	531279

Date:

Place: Nashik

Subject incharge **Head of Department** **Principal**

Seal Of Institute

Maharashtra State Board of Technical Education,Mumbai

Sir. Dr.M.S.Gosavi Polytechnic Institute Nashik Road

Certificate

This is to certify that following students of **THIRD SEMISTER** Diploma Engineering Program in Computer Engineering have successfully completed the Micro-Project entitled "**Blood Bank Management System**" under my supervision, in the partial fulfillment of Course **OOP (313304)** for Academic Year **2026-27** as per prescribed in the **MSBTE "K Scheme" curriculum.**

SR. No	Roll no.	Name of Student	Enrollment No.	Seat No.
02	17	PATIL PRATHMESH JITENDRA	25612590018	531267

Date:

Place: Nashik

Subject incharge **Head of Department** **Principal**

Seal Of Institute

Maharashtra State Board of Technical Education,Mumbai

Sir. Dr.M.S.Gosavi Polytechnic Institute Nashik Road

Certificate

This is to certify that following students of **THIRD SEMISTER** Diploma Engineering Program in Computer Engineering have successfully completed the Micro-Project entitled "**Blood Bank Management System**" under my supervision, in the partial fulfillment of Course **OOP (313304)** for Academic Year **2026-27** as per prescribed in the **MSBTE "K Scheme" curriculum.**

SR. No	Roll no.	Name of Student	Enrollment No.	Seat No.
03	12	PRAVIN CHETAN PATIL	25612590013	531262

Date:

Place: Nashik

Subject incharge **Head of Department** **Principal**

Seal Of Institute

Maharashtra State Board of Technical Education,Mumbai

Sir. Dr.M.S.Gosavi Polytechnic Institute Nashik Road

Certificate

This is to certify that following students of **THIRD SEMISTER** Diploma Engineering Program in Computer Engineering have successfully completed the Micro-Project entitled "**Blood Bank Management System**" under my supervision, in the partial fulfillment of Course **OOP (313304)** for Academic Year **2026-27** as per prescribed in the **MSBTE "K Scheme" curriculum.**

SR. No	Roll no.	Name of Student	Enrollment No.	Seat No.
04	20	NIKAM ROHIT ANIL	25612590021	531270

Date:

Place: Nashik

Subject incharge **Head of Department** **Principal**

Seal Of Institute

INDEX

Sr. No	Title	Page No.
1	INTRODUCTION	06
2	OBJECT ORIENTED DESIGN & PRINCIPLES	06
3	SYSTEM ARCHITECTURE & FLOW	08
4	ALGORITHMS & LOGIC DESIGN	09
5	SOURCE CODE IMPLEMENTATION & UNIQUE COMMANDS	10
6	TESTING & EXECUTION	14
7	CONCLUSION	15
8	EVALUATION	15

1. INTRODUCTION

1.1 Project Overview

The Blood Bank Management System is a menu-driven console application developed in C++ as an object-oriented programming micro-project. It maintains a register of blood donors — capturing donor identity, age, contact number, blood group and city — and lets the operator add donors, list every registered donor, and instantly search donors by blood group. All records are stored permanently in a flat text file (blood_bank_data.txt), so the data survives between program runs without requiring any database software.

1.2 Purpose and Scope

Purpose — to replace the manual donor register of a small blood bank with a fast, reliable, file-backed computerised system. Scope — the delivered system covers donor registration, complete listing and blood-group search. The design is deliberately modular (Person, Donor and BloodBankManager classes), so patient management, stock inventory and compatibility-based issue operations can be added later without restructuring the existing code.

1.3 Problem Statement

During emergencies, finding a compatible donor quickly is a matter of life and death, yet many blood banks still rely on paper registers that are slow to search, easy to misplace and impossible to query. The problem this project solves is: design an object-oriented system that stores donor demographics together with blood-group information, retrieves matching donors within seconds, and preserves the records between sessions using file handling.

2. OBJECT ORIENTED DESIGN & PRINCIPLES

2.1 Application of OOP Concepts

The four pillars of object-oriented programming are applied throughout the project, as explained below.

2.1.1 Classes and Objects (Person, Donor, BloodBankManager)

Three classes form the core of the program. Person models any human being with name, age and contact number. Donor extends Person with donorID, bloodGroup and city. BloodBankManager is a service class that coordinates all operations. Real objects of these classes are created at runtime — for example the manager object inside main(), and the temporary Donor object d created inside addDonor().

2.1.2 Encapsulation and Data Hiding (private attributes in Donor)

All attributes of Donor are declared private, so they cannot be read or modified directly from outside the class. Access is provided only through controlled public member functions — acceptDonorDetails(), displayDonorDetails(), getBloodGroup(), getDonorID() and saveToFile(). This separates internal state from the outside world, prevents invalid values from being assigned blindly, and lets the internal representation change without affecting other modules.

2.1.3 Inheritance (Donor publicly inheriting from Person)

Donor is derived from Person using public inheritance (class Donor : public Person). Every donor therefore ‘is a’ person: the fields name, age and contact and the methods getPersonData() / showPersonData() are reused instead of being re-declared. acceptDonorDetails() calls the base-class function getPersonData() to fill the inherited portion of the object — a practical demonstration of code reuse through inheritance.

2.1.4 Polymorphism and Method Overriding

The inheritance hierarchy lays the foundation for polymorphism: a Person reference or pointer can address a Donor object and invoke its shared interface. In the present design the member functions are resolved at compile time; declaring showPersonData() as virtual would enable runtime (dynamic) binding, so a later Patient class derived from Person could override it and one base-class loop could display Person, Donor or Patient objects through a single interface. This is the natural extension point of the project toward a fuller blood-bank system with multiple user types.

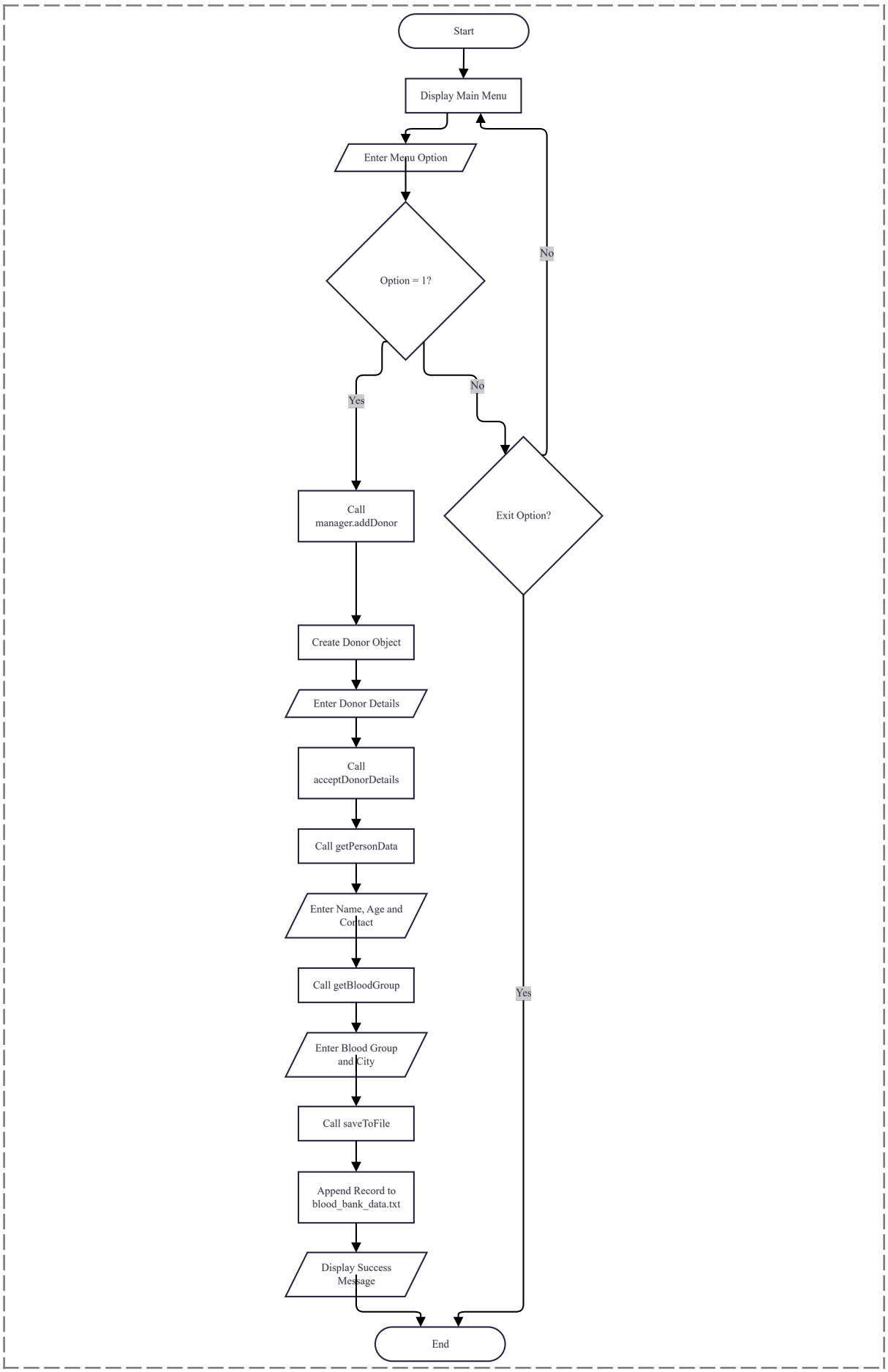
2.2 FLOWCHART

Use Case Diagram — the actor is the blood-bank operator (admin). The use cases are Add New Donor, View All Donors, Search Donors by Blood Group and Exit, all reached through the menu interface.

Class Diagram — Person sits at the top of the hierarchy, Donor inherits from it, and BloodBankManager uses Donor objects to perform operations. The table below lists every class with its attributes and methods.

Class	Key Attributes	Key Methods
Person	name, age, contact (protected)	getPersonData(), showPersonData()
Donor : public Person	donorID, bloodGroup, city (private)	acceptDonorDetails(), displayDonorDetails(), saveToFile(), getBloodGroup(), getDonorID()
BloodBankManager	— (uses Donor objects)	addDonor(), viewAllDonors(), searchByBloodGroup()

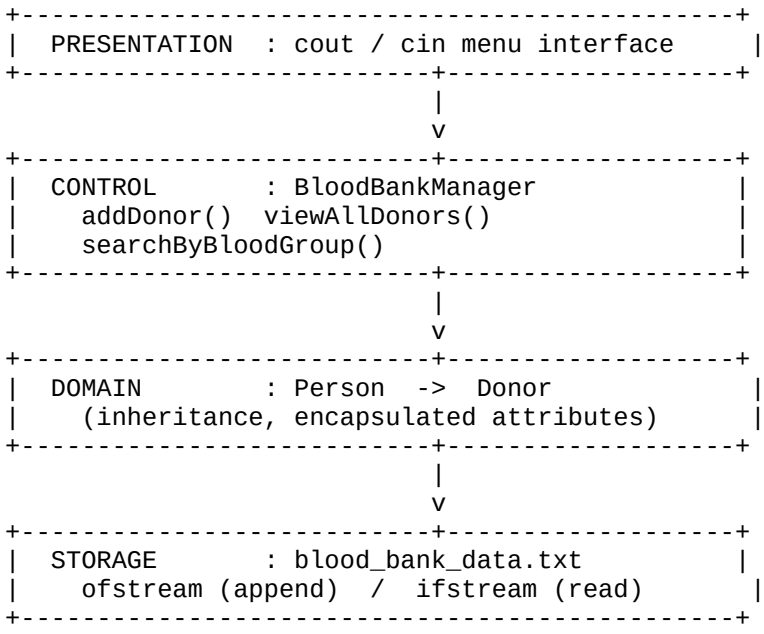
Sequence Diagram (Add New Donor) — operator chooses menu option 1 → main() calls manager.addDonor() → a Donor object collects details through acceptDonorDetails() → base-class getPersonData() reads name, age and contact → saveToFile() appends the formatted record to blood_bank_data.txt → the success message returns up the chain.


Start
Display Main Menu
Enter Menu Option
Option = 1?
No
Yes
No
Exit Option?
Call manager.addDonor
Create Donor Object
Enter Donor Details
Call acceptDonorDetails
Call getPersonData
Enter Name, Age and Contact
Call getBloodGroup
Yes
Enter Blood Group and City
Call saveToFile
Append Record to blood_bank_data.txt
Display Success Message
End

3. SYSTEM ARCHITECTURE & FLOW

3.1 Architecture Workflow

The system is organised in four cooperating layers. The do-while menu loop is the entry point, the switch statement dispatches the operator's choice to the manager object, and the manager methods either create a Donor (write path) or read the data file record by record (read path):



3.2 Entity Relationship and Data Flow

The system has one entity — DONOR — with attributes donorID (primary identifier), name, age, contact, bloodGroup and city. Data flows in two directions: the write path (keyboard → Donor object → ofstream append → blood_bank_data.txt) and the read path (blood_bank_data.txt → ifstream extraction loop → console). Each line of the file is a space-separated record in the order:

```
donorID  name  age  contact  bloodGroup  city
```

```
[ Admin input ] -> Donor object -> ofstream (append) -> blood_bank_data.txt
blood_bank_data.txt -> ifstream (>>) -> record loop -> [ Console output ]
```

4. ALGORITHMS & LOGIC DESIGN

4.1 Blood Group Compatibility & Matching Algorithm (searchByBloodGroup)

searchByBloodGroup() implements matching by exact blood group: the target group is compared with every stored record and all matches are printed. Transfusion medicine, however, also allows compatible substitutions, based on the donor / recipient relationships below — the basis for extending the search from same-group matching to compatibility matching:

Donor Group	Can donate to

O-	O-, O+, A-, A+, B-, B+, AB-, AB+ (universal donor)
O+	O+, A+, B+, AB+
A-	A-, A+, AB-, AB+
A+	A+, AB+
B-	B-, B+, AB-, AB+
B+	B+, AB+
AB-	AB-, AB+
AB+	AB+ (universal recipient)

Logic of the implemented search (single sequential pass):

```
OPEN  blood_bank_data.txt        ; if open fails -> print message, RETURN
READ  targetGroup                ; blood group entered by operator
WHILE (file >> id >> name >> age >> contact >> bGroup >> city)
    IF bGroup = targetGroup THEN
        PRINT donor details ; found = true
IF found = false THEN PRINT "No donors found ..."
CLOSE file
```

4.2 Donor Search & Validation Algorithm

The search is a single-pass sequential scan — O(n) in the number of donors — and never loads the whole file into memory, so it works even for very large registers. Validation is handled at three points: the file-open check prevents a crash when no data file exists yet; the stream extraction loop stops automatically at end-of-file or at a malformed record; and the found flag guarantees meaningful feedback to the operator. On the entry side, typed input (integer donorID / age, string blood group and city) keeps the record structure consistent for every later read.

4.3 Inventory Threshold & Alert Algorithm

Designed as the stock-management extension of the system (future scope), the alert algorithm watches the number of units available for every blood group:

```
THRESHOLD = 10                       ; minimum safe stock
FOR each group IN {A+, A-, B+, B-, O+, O-, AB+, AB-}
    units = COUNT of units of that group in the inventory file
    IF units < THRESHOLD THEN
        PRINT "ALERT: blood group", group, "below threshold"
        PRINT "units remaining:", units
```

The check runs after every issue operation so that the bank is warned before any group runs critically low.

5. SOURCE CODE IMPLEMENTATION & UNIQUE COMMANDS

5.1 Core Classes (Person and Donor)

Person holds the protected demographic data and provides getPersonData() for input and showPersonData() const for display. Donor adds the private fields donorID, bloodGroup

and city, the constructor that initialises them, the combined entry/display functions and the const getters getBloodGroup() and getDonorID() used by the search module.

5.2 Donor Management Module (addDonor, viewAllDonors)

BloodBankManager::addDonor() creates a Donor object on the stack, fills it through acceptDonorDetails() and stores it with saveToFile(). viewAllDonors() opens the data file and prints every record through the extraction loop, with a clear message when no records exist yet.

5.3 File Handling Module (saveToFile using ofstream and ifstream)

saveToFile() opens blood_bank_data.txt in append mode (ofstream with ios::app) so each new donor is added at the end without overwriting earlier records; the stream is closed explicitly after writing. Both view and search open the same file with ifstream and read formatted fields using the extraction operator >> inside the while condition, which also detects end-of-file. A failure branch reports an error if the file cannot be opened.

5.4 Unique Functions, Operators and Code Snippets Used

- cin.ignore() discards the newline left in the input buffer by cin >> so that the following getline reads the full name correctly.
- getline(cin, name) reads names containing spaces in one line.
- ios::app (append flag) preserves all previous donor records every time a new one is written.
- Stream extraction operator >> reads space-separated fields directly from the file: while (inFile >> id >> name >> age >> contact >> bGroup >> city)
- protected access specifier makes name / age / contact invisible to the outside world yet available to the derived class Donor.
- const member functions (showPersonData(), displayDonorDetails(), saveToFile(), getBloodGroup()) promise not to modify the object.
- do-while loop keeps the menu running at least once; switch-case dispatches choices 1 –4 with a default branch for invalid input.

Key snippets from the implementation:

```
class Donor : public Person { ... };      // inheritance
cin.ignore();  getline(cin, name);        // correct string input
ofstream outFile("blood_bank_data.txt", ios::app);   // append
while (inFile >> id >> name >> age >> contact >> bGroup >> city)
    if (bGroup == targetGroup) { ... }    // search match
```

5.5 Main Application Driver / Menu Interface

main() creates one BloodBankManager object and runs the do-while menu. The switch statement forwards each valid choice to the corresponding manager function, choice 4 exits with a farewell message, and the default branch reports an invalid choice:

```
do {
    cout << "1. Add New Donor";
    cout << "2. View All Donors";
    cout << "3. Search Donors by Blood Group";
    cout << "4. Exit";
    cin >> choice;
```

```
        switch (choice) { case 1: manager.addDonor(); break; ... }
    } while (choice != 4);
```

5.6 Complete Source Code

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>

using namespace std;

// Base Class demonstrating Inheritance
class Person {
protected:
    string name;
    int age;
    string contact;

public:
    // Constructor
    Person() {
        name = "";
        age = 0;
        contact = "";
    }

    void getPersonData() {
        cout << "Enter Name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter Age: ";
        cin >> age;
        cout << "Enter Contact Number: ";
        cin >> contact;
    }

    void showPersonData() const {
        cout << "Name    : " << name << endl;
        cout << "Age     : " << age << endl;
        cout << "Contact : " << contact << endl;
    }
};

// Derived Class demonstrating Encapsulation & Inheritance
class Donor : public Person {
private:
    int donorID;
    string bloodGroup;
    string city;

public:
    Donor() {
        donorID = 0;
        bloodGroup = "";
        city = "";
    }

    // Function to add donor details
    void acceptDonorDetails() {
        cout << "\n--- Enter Donor Information ---\n";
        cout << "Enter Donor ID: ";
        cin >> donorID;
        getPersonData(); // Calling base class method
        cout << "Enter Blood Group (e.g., A+, O-, B+): ";
```

```
        cin >> bloodGroup;
        cout << "Enter City: ";
        cin >> city;
    }

    // Function to display donor details
    void displayDonorDetails() const {
        cout << "-----------------------------------\n";
        cout << "Donor ID    : " << donorID << endl;
        showPersonData();
        cout << "Blood Group : " << bloodGroup << endl;
        cout << "City        : " << city << endl;
        cout << "-----------------------------------\n";
    }

    // Getter for blood group search
    string getBloodGroup() const {
        return bloodGroup;
    }

    int getDonorID() const {
        return donorID;
    }

    // Save donor record to file (File Handling)
    void saveToFile() const {
        ofstream outFile("blood_bank_data.txt", ios::app);
        if (outFile.is_open()) {
            outFile << donorID << " " << name << " " << age << " "
                    << contact << " " << bloodGroup << " " << city << endl;
            outFile.close();
        } else {
            cout << "Error opening file for writing!\n";
        }
    }
};

// Blood Bank Manager Class (Managing collections of donors)
class BloodBankManager {
public:
    void addDonor() {
        Donor d;
        d.acceptDonorDetails();
        d.saveToFile();
        cout << "\n[Success] Donor added and saved to file successfully!\n";
    }

    void viewAllDonors() {
        ifstream inFile("blood_bank_data.txt");
        if (!inFile) {
            cout << "\nNo donor records found!\n";
            return;
        }

        int id, age;
        string name, contact, bGroup, city;

        cout << "\n=======================================\n";
        cout << "          ALL REGISTERED DONORS         \n";
        cout << "=======================================\n";

        while (inFile >> id >> name >> age >> contact >> bGroup >> city) {
            cout << "Donor ID    : " << id << endl;
            cout << "Name        : " << name << endl;
```

```
            cout << "Age         : " << age << endl;
            cout << "Contact     : " << contact << endl;
            cout << "Blood Group : " << bGroup << endl;
            cout << "City        : " << city << endl;
            cout << "----------------------------------------\n";
        }
        inFile.close();
    }

    void searchByBloodGroup() {
        string targetGroup;
        cout << "\nEnter Blood Group to Search (e.g., A+, O+): ";
        cin >> targetGroup;

        ifstream inFile("blood_bank_data.txt");
        if (!inFile) {
            cout << "\nNo records found to search!\n";
            return;
        }

        int id, age;
        string name, contact, bGroup, city;
        bool found = false;

        cout << "\n--- Search Results for Blood Group: " << targetGroup << "
---\n";
        while (inFile >> id >> name >> age >> contact >> bGroup >> city) {
            if (bGroup == targetGroup) {
                cout << "Donor ID    : " << id << endl;
                cout << "Name        : " << name << endl;
                cout << "Contact     : " << contact << endl;
                cout << "City        : " << city << endl;
                cout << "----------------------------------------\n";
                found = true;
            }
        }
        if (!found) {
            cout << "No donors found with blood group " << targetGroup << ".
\n";
        }
        inFile.close();
    }
};

// Main Function
int main() {
    BloodBankManager manager;
    int choice;

    do {
        cout << "\n========================================\n";
        cout << "   BLOOD BANK MANAGEMENT SYSTEM (OOP)   \n";
        cout << "========================================\n";
        cout << "1. Add New Donor\n";
        cout << "2. View All Donors\n";
        cout << "3. Search Donors by Blood Group\n";
        cout << "4. Exit\n";
        cout << "----------------------------------------\n";
        cout << "Enter your choice (1-4): ";
        cin >> choice;

        switch (choice) {
            case 1:
                manager.addDonor();
```

```
                break;
            case 2:
                manager.viewAllDonors();
                break;
            case 3:
                manager.searchByBloodGroup();
                break;
            case 4:
                cout << "\nThank you for using Blood Bank System. Exiting...
\n";
                break;
            default:
                cout << "\nInvalid choice! Please try again.\n";
        }
    } while (choice != 4);

    return 0;
}
```

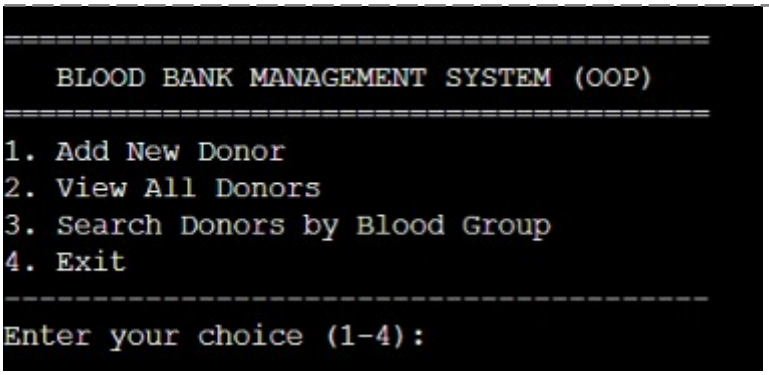
6. TESTING & EXECUTION

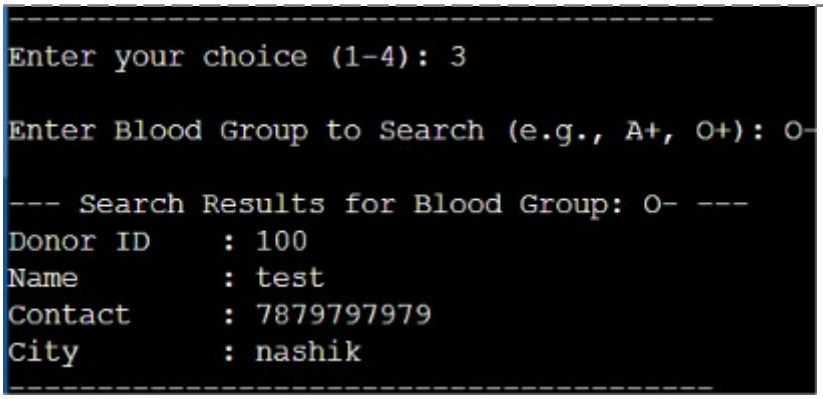
6.1 Test Cases and Scenarios

TC	Scenario	Steps / Input	Expected Result
01	Add donor	Menu 1; ID 101, Rahul, 25, 9876543210, A+, Nashik	Success message; record appended to blood_bank_data.txt
02	View all donors	Menu 2 after TC-01	All registered donors listed with full details
03	Search — match	Menu 3; blood group A+	Every A+ donor shown (ID, name, contact, city)
04	Search — no match	Menu 3; blood group AB- (none)	“No donors found with blood group AB-.”
05	View with no data	Menu 2 before any donor is added	“No donor records found!” (no crash)
06	Invalid choice	Menu 9	“Invalid choice! Please try again.”
07	Exit	Menu 4	Thank-you message; program terminates cleanly

6.2 Output Execution Screenshots

Console output of the menu, donor registration, listing and blood-group search, captured after compiling with the GNU C++ compiler:





7. CONCLUSION

The Blood Bank Management System was successfully designed, coded and tested in C++. The project demonstrates a practical application of object-oriented principles: inheritance and encapsulation are visible in the Person – Donor relationship, modularity in BloodBankManager, and durable storage in the file-handling code. All three menu operations work as specified, with graceful handling of missing data files and unsuccessful searches.

Future enhancements include compatibility-based donor search using the matrix in section 4.1, a patient-management module, blood-stock inventory with threshold alerts, and migration from flat files to a database for multi-user access.

8. EVALUATION

Marks section :

Roll No.	Enrollment no	Full Name of the Student.	Marks out of (6) Performed by the group.	Marks out of (4) Performed By oral Presentation.	Total out of (10).	
29	25612590030	ALAI YASH AMEY				
17	25612590018	PATIL PRATHMESH JITENDRA				
12	25612590013	PRAVIN CHETAN PATIL				
20	25612590021	NIKAM ROHIT ANIL				

Prof. N.B Ghati
Department of Computer Engineering
Sir Dr M.S Gosavi Polytechnic Institute, Nashik Road, Nashik-422101.